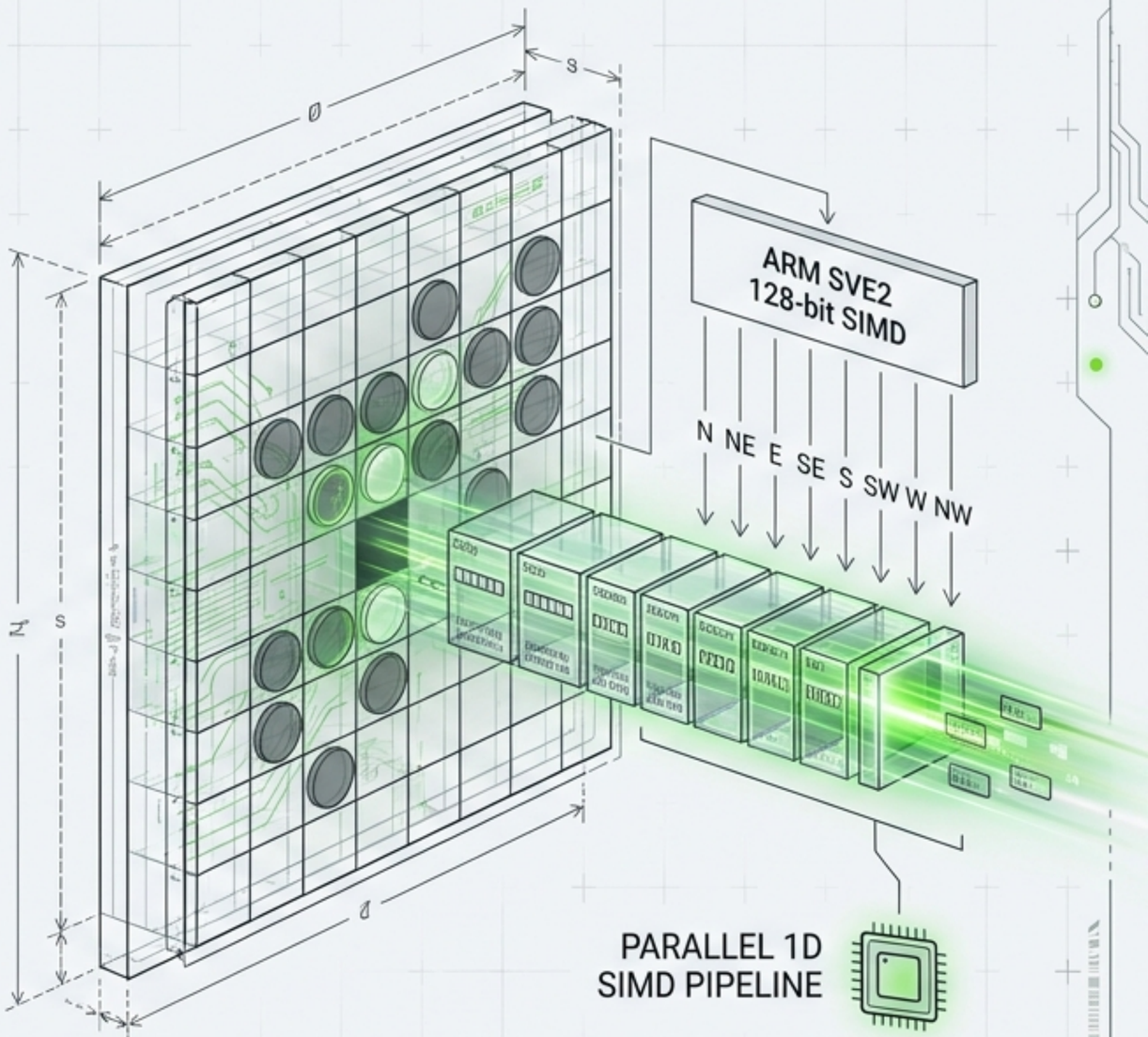
NVIDIA DGX Spark: Mastering Othello Legal Move Generation

ARM SVE2 128-bit SIMDを用いた「合法手生成の8方向同時探索」完全解剖

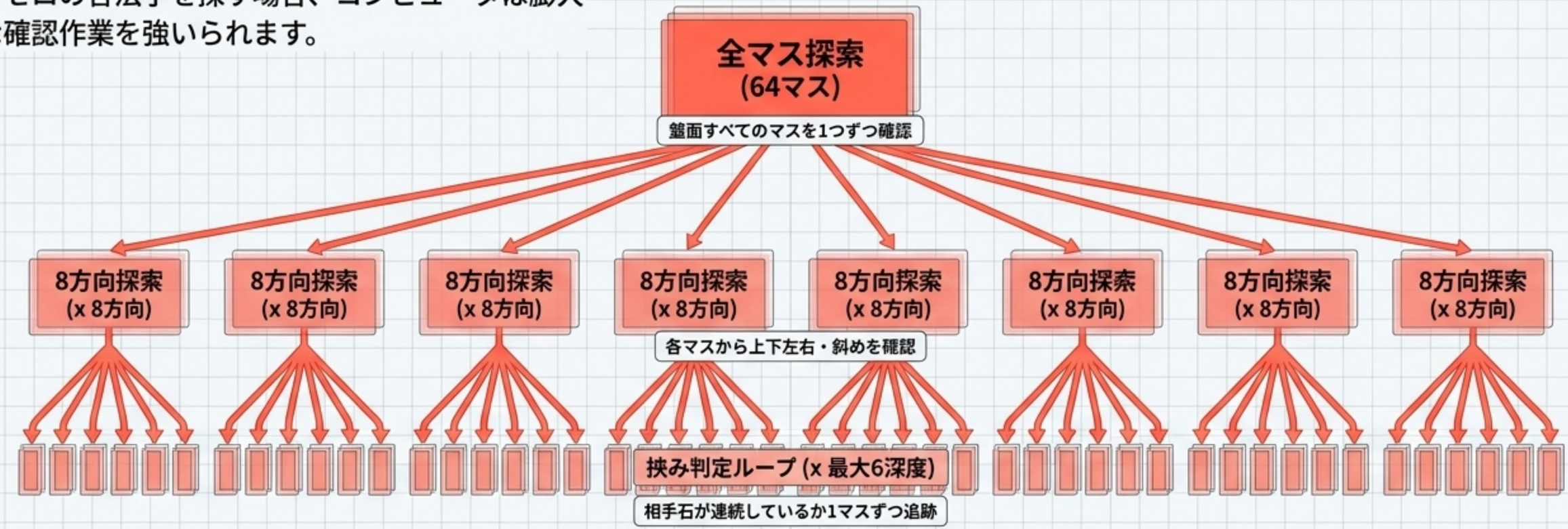
2Dの盤面問題を1Dのベクトル演算へ次元圧縮し、
ネストされたループを完全に排除するアルゴリズム設計の極意。



NotebookLM

スカラー処理の限界：巨大なネストループの壁

従来のプログラミング手法（スカラー処理）でオセロの合法手を探す場合、コンピュータは膨大な確認作業を強いられます。



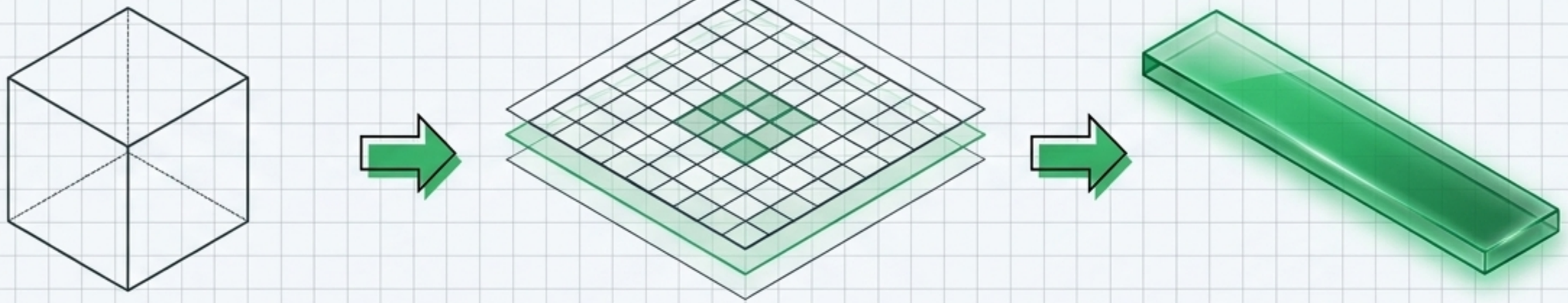
64 × 8 × 6 = 最悪 3,072回

この分岐と反復の連続が、思考エンジンの致命的なボトルネック（処理速度の壁）となります。

NotebookLM

SVE2による野望：「次元圧縮」による完全並列化

ARM SVE2の128ビットレジスタを活用し、3次元のネストループを「1つの並列パイプライン」へと平坦化します。
本資料では、この進化を3つの次元で解き明かします。



1st Dimension (Logic)
Shift + AND
[1方向・1石の最適化]

2nd Dimension (Bitboard)
全自石の一斉シフト
[64マスの同時処理]

3rd Dimension (SIMD)
SVE2並列処理
[8方向の同時処理]

最終目標：条件分岐（if文）をゼロにし、ベクトル演算のみで盤面全域の合法手を瞬時に炙り出す。

NotebookLM

1st Dimension: ビット演算による探索の3原則

オセロのルール「自石から相手石が連続し、その先に空きマスがある」という条件は、
以下の3つの論理演算に完全翻訳されます。

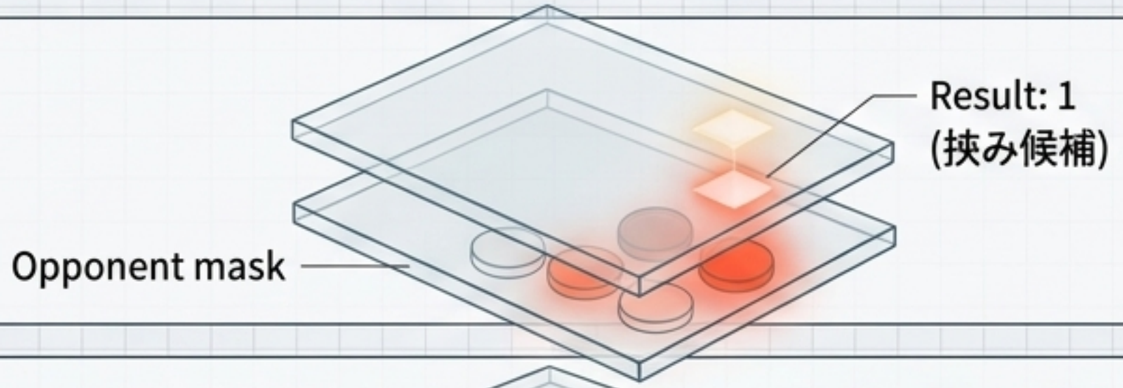
原理① シフト（移動）

操作: Shift
意味: 「隣のマスに進む」。右方向なら1ビットシフト、
下方向なら8ビットシフト。



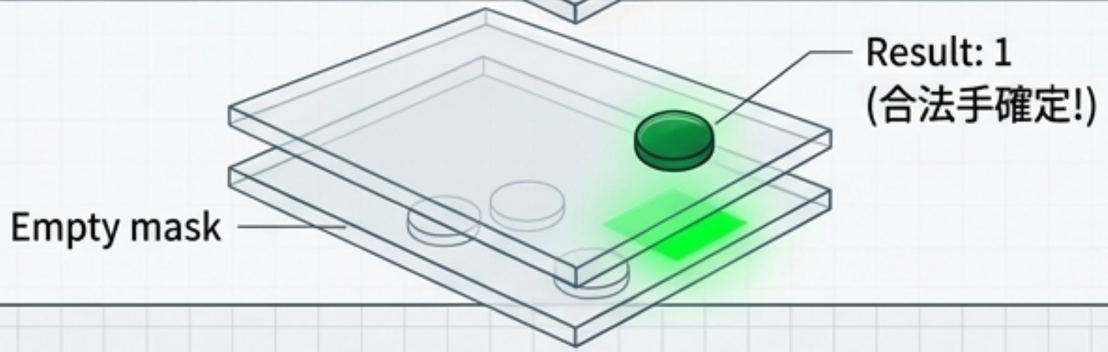
原理② AND（相手石の判定）

操作: AND(opp)
意味: 「進んだ先に相手石があるか」。結果が1なら挟み
候補として蓄積、0なら探索終了。



原理③ AND（空きマスの確認）

操作: AND(empty)
意味: 「相手石列の先が空きマスか」。結果が1なら、
そこが合法手（石を置けるマス）として確定。



NotebookLM

Shift + AND の軌跡：成功と失敗のメカニズム

成功例 (合法手発見)

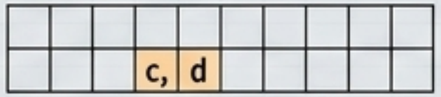
	a	b	c	d	e	f	g	h	h	
my	0	1	0	0	0	0	0	0	0	bに黒石
opp	0	0	1	1	1	0	0	0	0	c, d, eに白石
empty	1	0	0	0	0	0	1	1	1	a, f, g, hが空き

① shift(my, 右1) → b→c

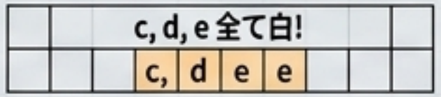


② AND(opp) → c=1(白あり!) → 挟み候補スタート

③ shift(cand, 右1)→c→d, AND(opp)→d=1 → 候補拡張



④ shift(cand, 右1)→d→e, AND(opp)→e=1 → さらに拡張



⑤ shift(cand, 右1) AND empty → 相手石列の先が空きマスか？



f=1: 合法手！

- Step 1: 自石(b)を右にシフト(b→c)。白石(opp)とAND=1。挟み候補スタート。
- Step 2: さらにシフト(c→d→e)。白石が3マス連続と判明。
- Step 3: さらにシフト(e→f)。空きマス(empty)とAND=1。
- 結果: fマスを合法手として確定！

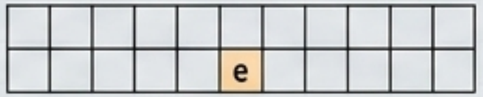
f が合法手！fに黒を置くとc, d, eの白石を挟める

失敗例 (自石に到達しない)

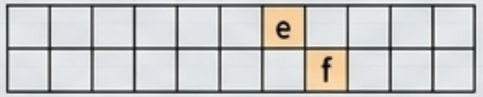
	a	b	c	d	e	f	g	h	h	
empty	0	0	0	1		0	0	0	0	dが空き
opp	0	0	0	0	0	1	1	0	0	e, fに白石
my	0	1	0	0	0	0	0	0	0	bに黒石

dから右方向探索: d→e(白)→f(白)→g(空) = 自石がない!

① cursor=d→eにシフト, AND(opp)=1 → 相手石あり、続行



② cursor=e→fにシフト, AND(opp)=1 → 相手石あり、続行



③ cursor=f→gにシフト, AND(opp)=0 → 相手石途切れた！



④ gに自石があるか確認: AND(my)=0 → 自石もない！



- Step 1-2: 相手石を右にシフトして追跡(d→e→f)。
- Step 3: さらにシフト(f→g)。相手石が途切れる(AND(opp)=0)。
- Step 4: そのマスに自石がないか確認(AND(my)=0)。
- 結果: 挟めないため合法手にはならない。

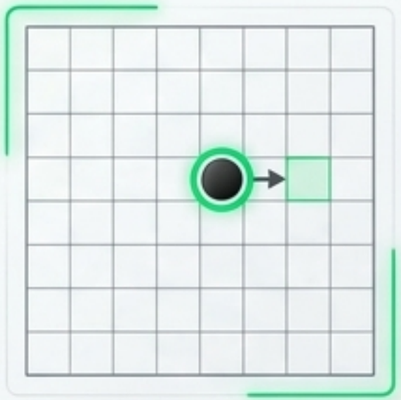
自石に到達しなかった → 白石を挟めない → dは合法手ではない

NotebookLM

2nd Dimension: ビットボードによる「全自石の一斉シフト」

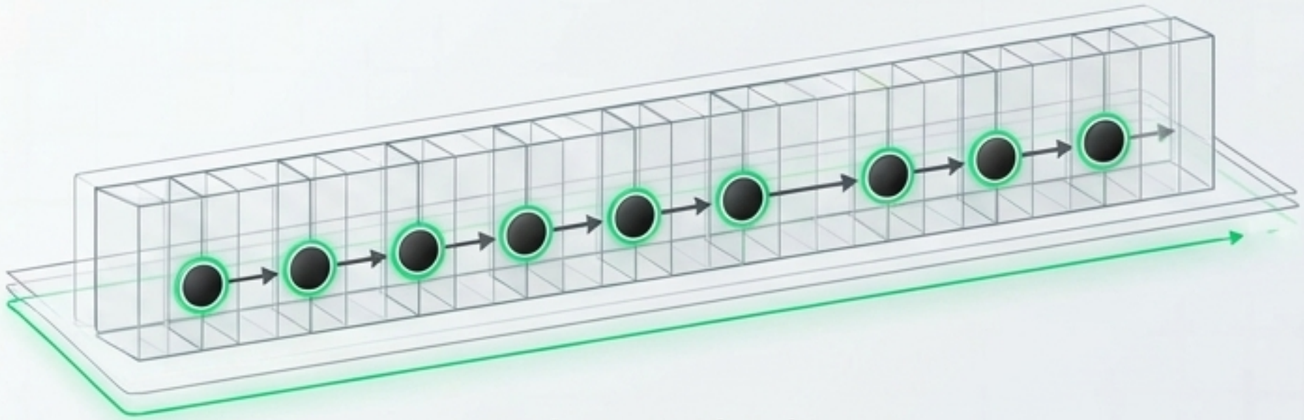
1マスずつ探索する素朴な方法から脱却します。
盤面全64マスを「1つの64ビット整数」に格納することで、演算の概念が根底から覆ります。

素朴な探索



盤面上の一つの黒石を選び、
右隣を確認する。

ビットボード探索



svdup_u64(my)
全ての黒石(my)を右に1ビットシフトする。

Key Insight

盤面上のすべての自石が同時に1マス進みます。このシフト結果と相手石マスクのANDを取ることで、「どの自石の右隣に相手石があるか」を全64マス分、1サイクルの命令で一斉判定します。

NotebookLM

同時並列で進む探索ルート

全自石の一斉シフトにより、異なる起点からの探索が干渉することなく並列に進みます。
自石が何個あっても、演算回数は増えません。

	a	b	c	d	e	f	g	a	b	c	d	f	g	h
my (自石)	1	0	0	0	0	0	0	1	0	0	0	0	0	0
opp (相手石)	0	0	0	1	1	1	0	0	0	1	1	1	0	0
empty	0	1	0	0	0	0	0	1	0	0	0	0	0	1

起点: a列とe列にある2つの黒石を同時に右へシフト。

Step 1: my全体を右に1シフト → shifted & AND opp (全自石の右隣に相手石があるか?)

my 1 b

a→b: shifted = 1 at b
AND opp

opp 0

Step 1: a列の石は右隣(b列)が空きマスのため即座に脱落 (AND(opp)=0)。

e 1

e→f: shifted = 1 at f
AND opp

opp 1

Step 2: e列の石は右隣(f列)に白石があるため探索継続 (AND(opp)=1)。

Step 2: candをさらに右に1シフト → shifted & AND opp

f→g: shifted = 1 at g
AND opp

opp 1

Step 3: e列からの探索のみがg列(白)→h列(空き)と進み、合法手を発見。

Step 3: candをさらに右に1シフト → AND empty

g→h h

g→h: shifted = 1 at h
AND empty

empty 1

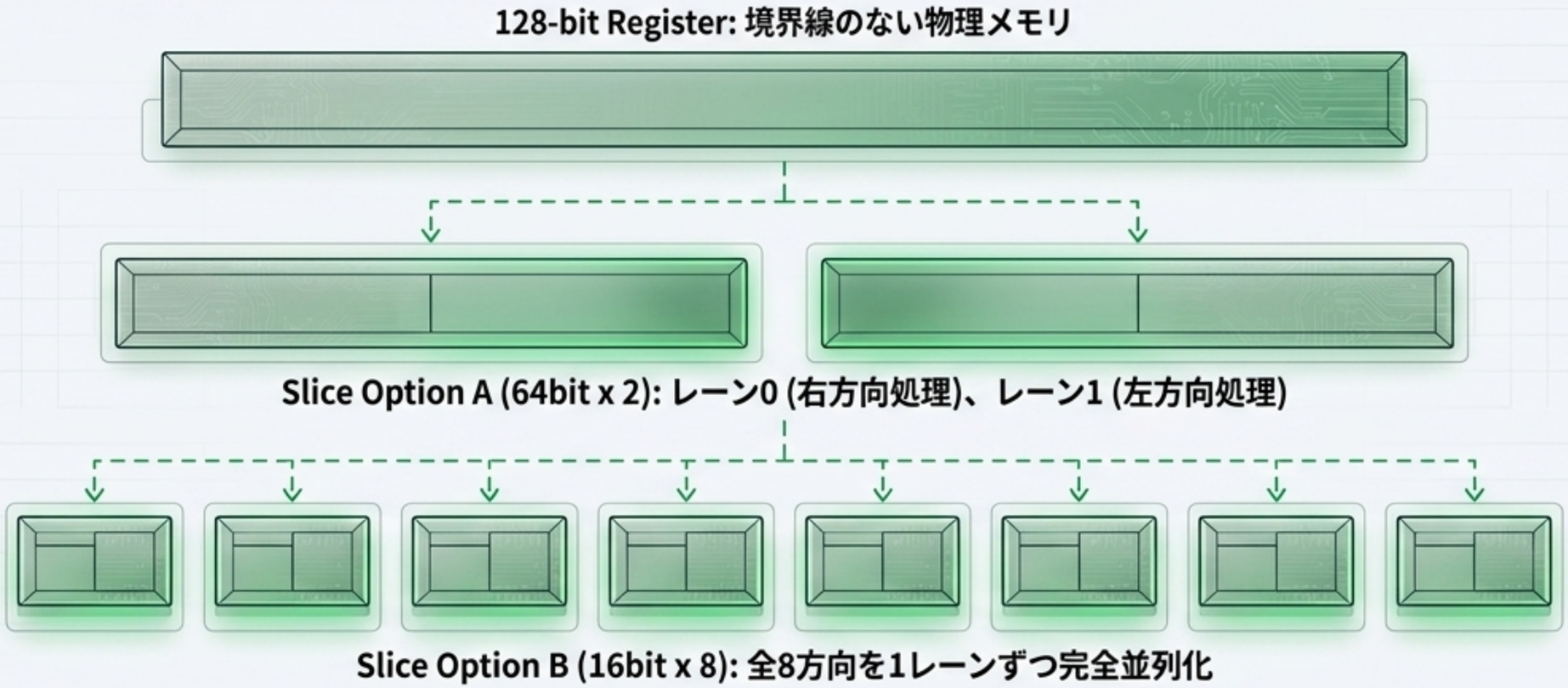
h=1: 合法手!

ループ回数は「盤面にある自石の数」ではなく、「相手石が最大何マス連続するか（最大6マス）」にのみ依存します。

NotebookLM

3rd Dimension: SVE2レジスタの「レーン」分割

DGX Sparkが誇るSVE2の真骨頂。1本の128ビット物理レジスタを、等幅の区画「レーン」に仮想的に分割し、全く独立した演算を同時実行します。

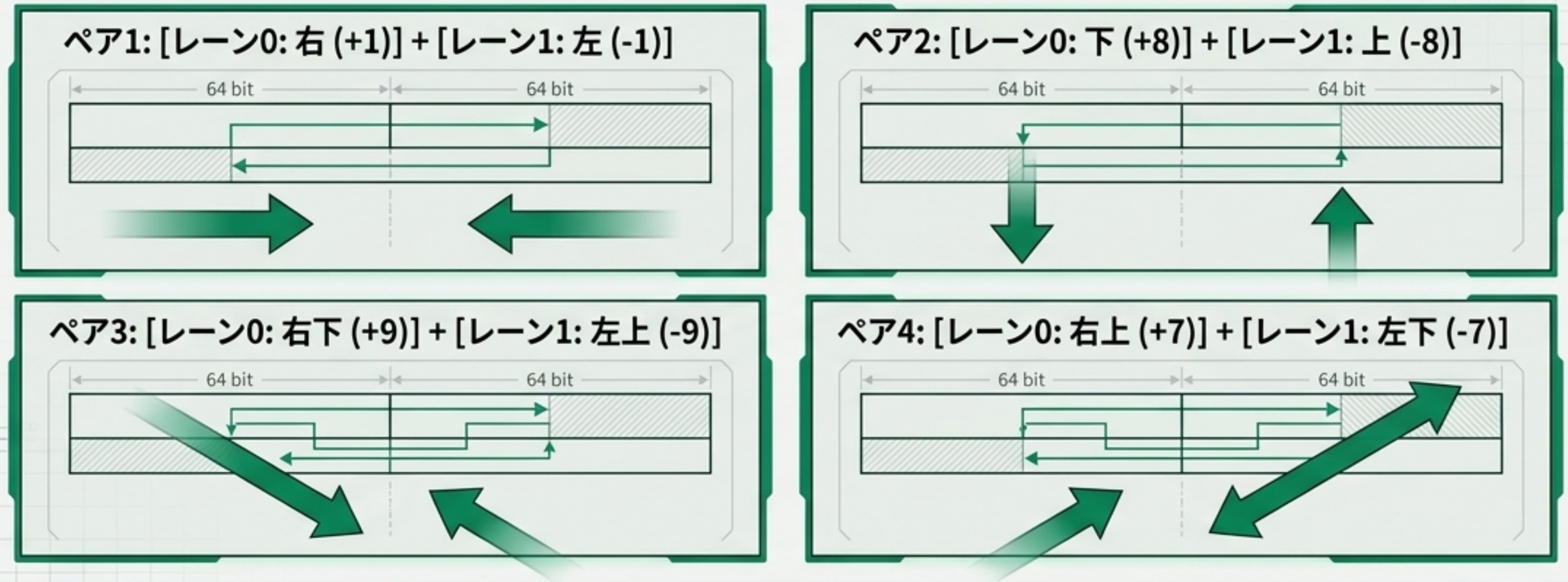


TECH NOTE:
レーン幅はハードウェアではなく「命令のサフィックス (例: _u64, _s16)」によってソフトウェア側から動的に指定されます。

NotebookLM

64bit×2レーンによる「逆方向ペア」の並列処理

本アルゴリズムでは、128ビットレジスタを64ビット×2レーンに分割して使用します。
8方向を「逆向きの2方向」ずつ4ペアに分け、1つのSIMD命令で同時処理します。



IMPACT BOX

svlsr (右シフト) と svlsl (左シフト) を各ペアに対して実行。
8回の方向ループを4回に半減させる劇的な最適化です。

NotebookLM

制御の要①：回り込みのハザード（The Wrap-Around Problem）

2次元の盤面を1次元の64ビット整数にマッピングしたことで、深刻な副作用が生じます。
端のマスをシフトすると、物理的にあり得ない場所へ「ワープ」してしまうのです。



- 現在地: h列のマス
  （例: h1 = Bit 7）
- 右へシフト (+1):
  Bit 7 + 1 = Bit 8
- ワープ発生:
  Bit 8 は次行の左端 (a2) に該当！

Problem Statement

h列の石を右にシフトすると、次行のa列に回り込む。
この「偽のビット」を除去しなければ、誤った合法手が生成されてしまいます。

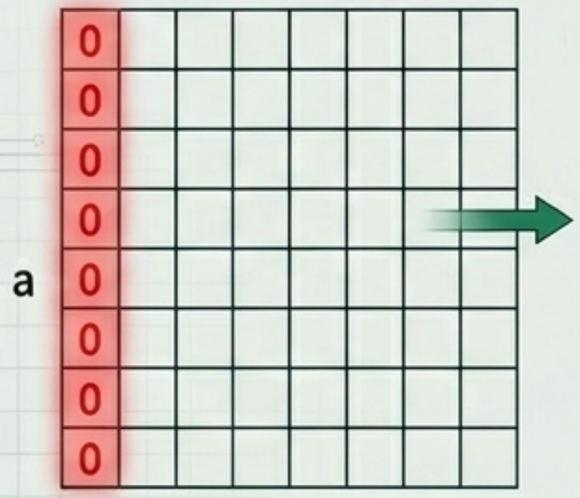
NotebookLM

制御の要②：EDGE_MASKによる「偽ビット」の遮断

回り込みを防ぐため、シフト方向の「出口となる辺」を強制的にゼロにする
特殊なマスク（ステンシル）を被せ、AND演算で不要なビットを消去します。

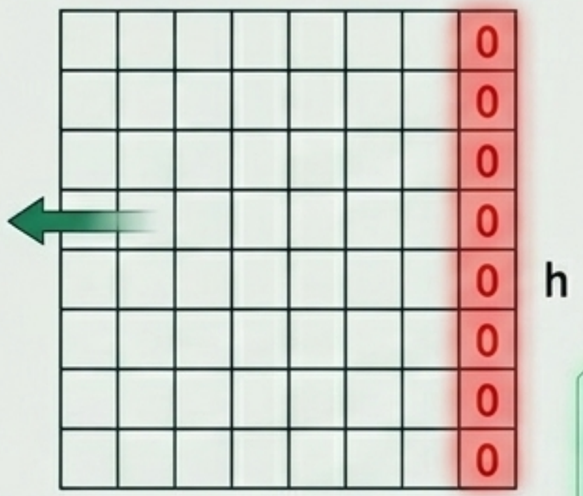
右方向シフト (+1):
a列（左端）に回り込んだビットを消去。

Mask: 0xFEFEFEFEFEFEFEFE
(各行のbit 0をゼロ化)



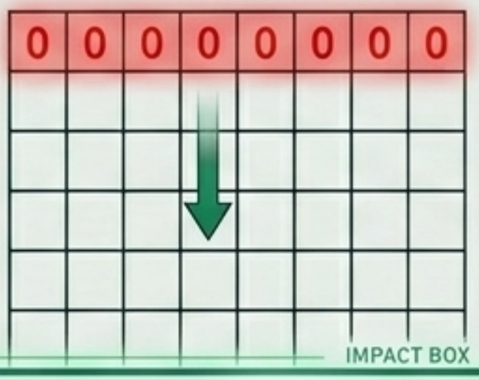
左方向シフト (-1):
h列（右端）に回り込んだビットを消去。

Mask: 0x7F7F7F7F7F7F7F7F
(各行のbit 7をゼロ化)



下方向シフト (+8):
1行目（上端）に溢れたビットを消去。

Mask: 0xFFFFFFFFFFFFFF00



IMPACT BOX

シフト後の結果に対して
svand_x(pg, shifted, v_mask) を適用し、
有効なシフト先だけを抽出します。

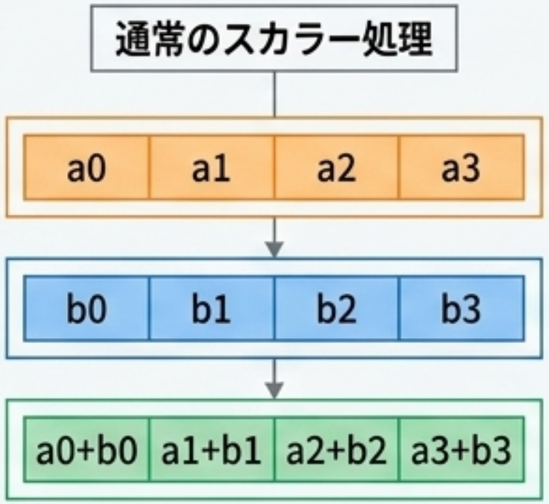
NotebookLM

制御の要③：述語（プレディケート）による絶対制御

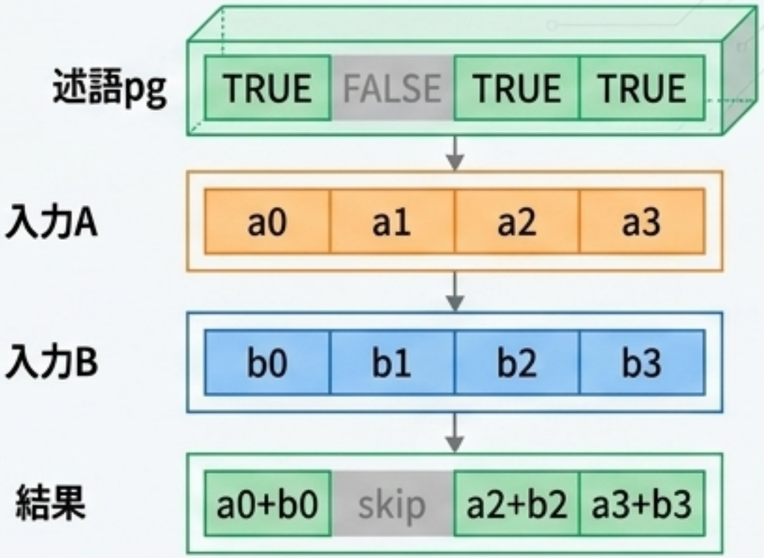
SVE2の真の力は、svbool_t（述語レジスタ）によるレーン単位の実行制御にあります。

通常のスカラー処理

「このデータを処理するか？」
を毎度 if 文で判定。
→ 分岐予測ミスの温床。



全レーンが必ず演算される



述語がFALSEのレーンはスキップ
→ 端数処理や条件分岐を排除

SVE2 述語制御 (svbool_t)

各レーンに TRUE / FALSE のマスクを付与。

THE _x ADVANTAGE

SVE2命令の _x サフィックフックス（例: svand_x）は、「FALSEレーンの結果は未定義でよい」という指示です。
ハードウェアは FALSE レーンの演算を完全にスキップするため、条件分岐を完全に排除しながら最高速で実行されます。

NotebookLM

SVE2アルゴリズム実装 [Step 1]: ベクトルの初期化

ここから、実際のアルゴリズム（generate_legal_moves_sve2）の動きを追います。
まずはスカラーデータを128ビットSIMD空間へ展開します。

`svptrue_b64()`

全64bitレーンを有効化（TRUE）する基準の述語を生成。



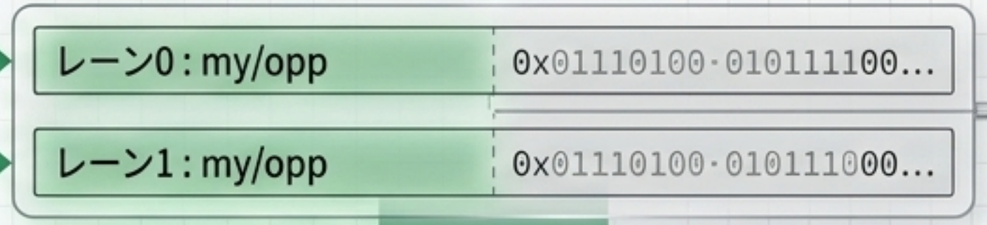
svdup_u64 全レーンに複製

`svdup_u64(my / opp)`

自石・相手石の単一ビットボードを、レーン0とレーン1の両方に複製（ブロードキャスト）。

スカラー (my/opp) = 0x...

0x0011110… 0111100…



svld1_u64 2方向のマスクを同時ロード

`svld1_u64(pg, &EDGE_MASK[d])`

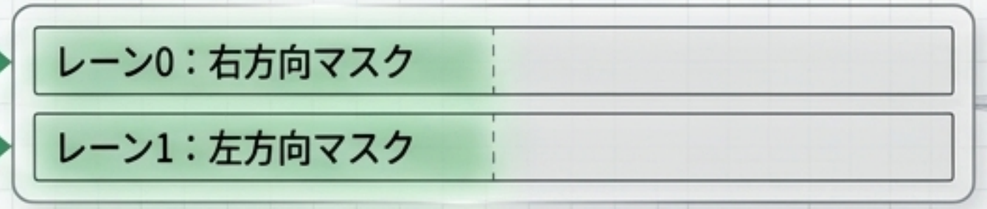
配列から2方向分（例：右用と左用）の端マスクを同時にロードし、ベクトルを準備。

EDGE MASK配列 [右方向]

0xFE11101… 0x31000…

EDGE MASK配列 [左方向]

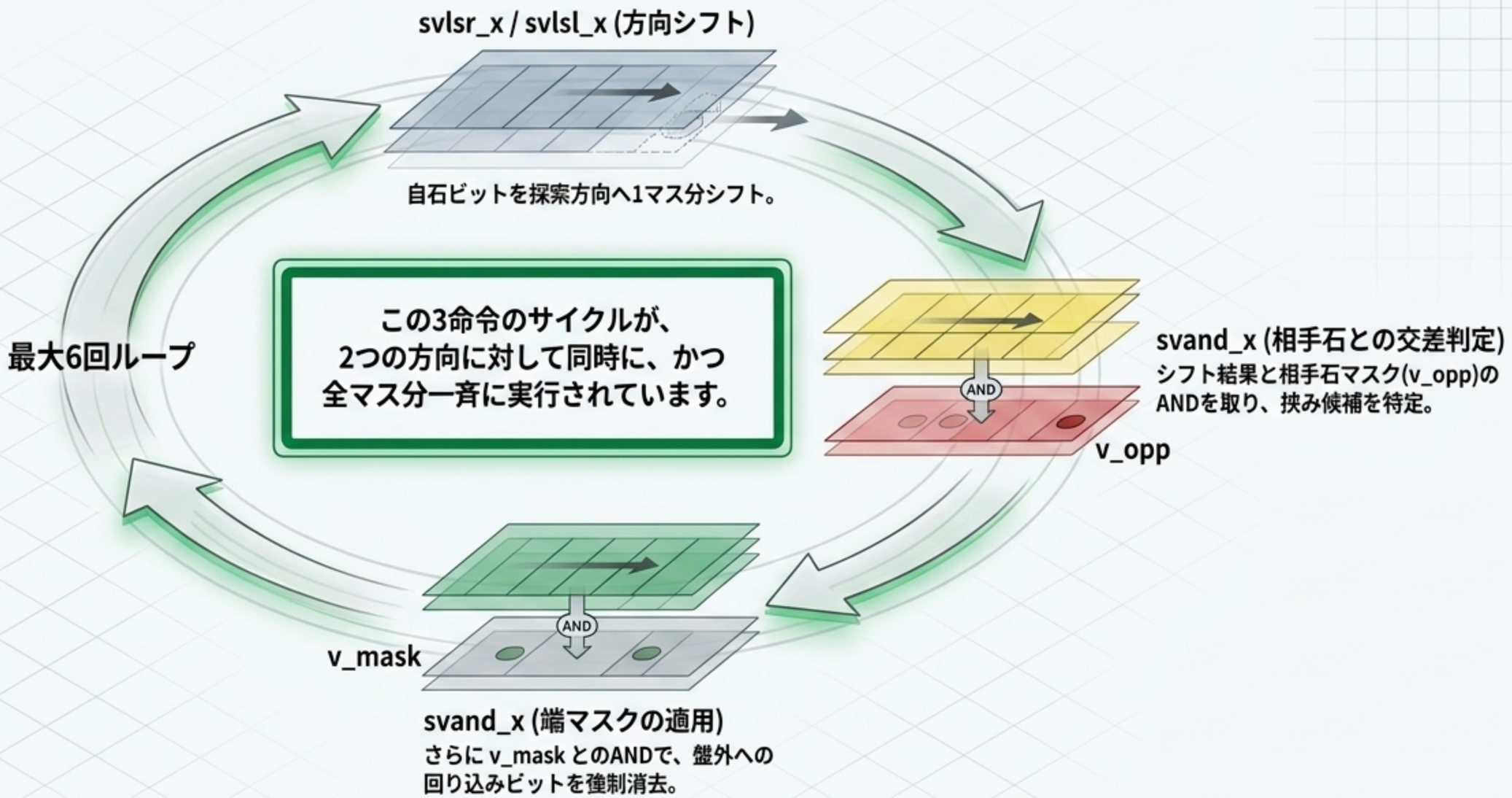
0x7F11011… 0x01011…



NotebookLM

SVE2アルゴリズム実装 [Step 2] : 挟み判定の連鎖ループ

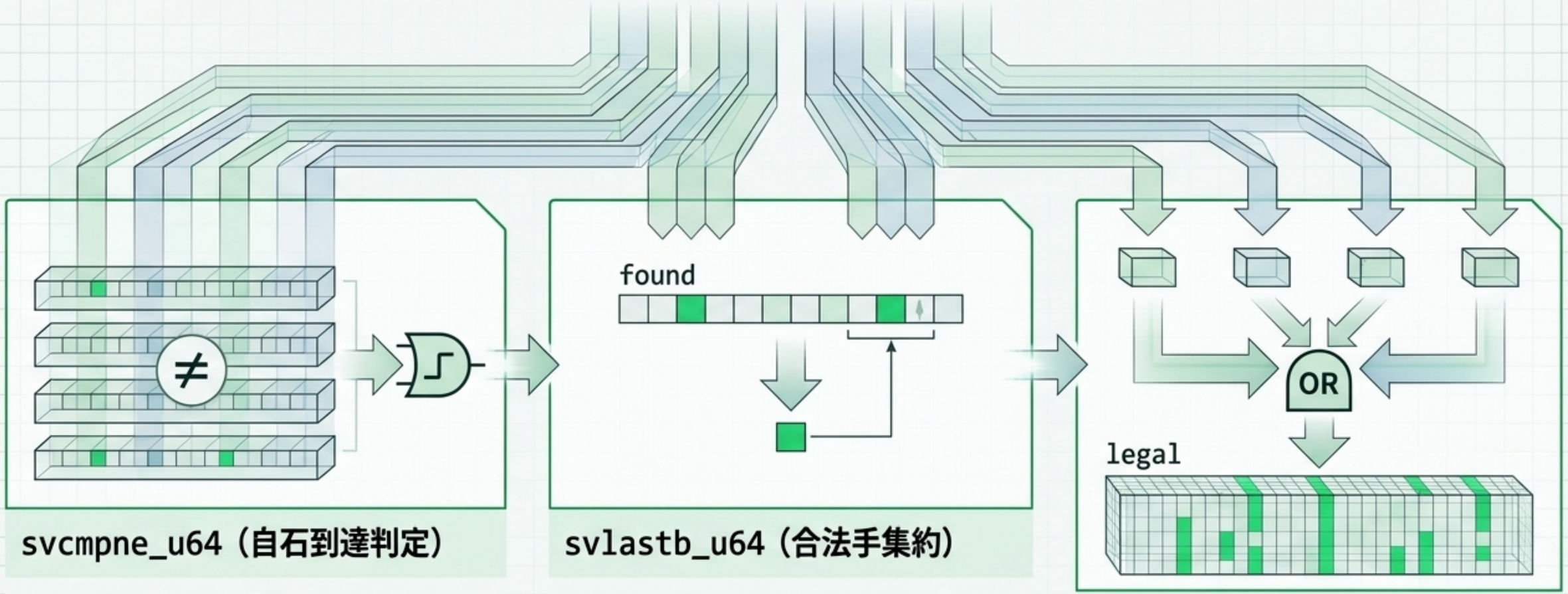
相手石が連続する限り、最大6回まで探索を拡張する心臓部のループ処理です。



NotebookLM

SVE2アルゴリズム実装 [Step 3]: 合法手の確定と集約

ループを抜けたら、「相手石連続の先が自石であるか（＝正しく挟めているか）」を検証し、結果をスカラー値に集約します。



svcmpne_u64（自石到達判定）

pgが有効な全レーンで「最終シフト位置 ≠ 0」を比較。「ビットが立っているか」を条件分岐なしに一斉判定し、新たな述語 found を生成。

svlastb_u64（合法手集約）

述語 found がTRUEとなったレーンの値をスカラーとして抽出。

legal |= ...（最終統合）

4ペア（全8方向）のビットマスクをORで統合。盤面上のすべての合法手を1つの64ビット整数として完成させます。

NotebookLM

結論：スカラーの壁を破壊する「3次元の並列化」

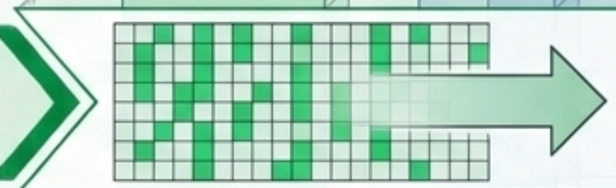
Chapter 3で解説したSVE2による合法手生成アルゴリズムは、計算の次元そのものを引き上げることで、圧倒的なスループットを実現します。

Dimension 1 (Logic):
Shift + AND



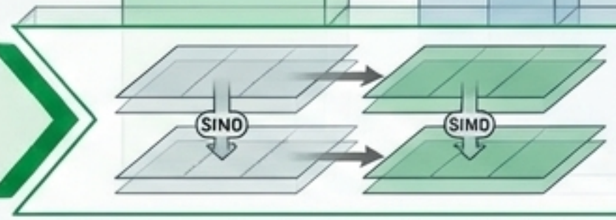
→ 複雑な条件判定を単一のビット演算へ還元。

Dimension 2 (Bitboard):
全自石の一斉シフト



→ 64マスの反復ループを「1回の演算」へ圧縮。

Dimension 3 (SIMD SVE2):
2レーン同時実行



→ 8方向のループを「4回のベクトル命令」へ半減。

[64マス × 8方向 × 6深度] の最悪3,072回のスカラー反復が、
[4ペア × 6ループ] のフラットな24回のSIMD命令群へと次元圧縮される。
これが、DGX Spark環境におけるSVE2プログラミングの真髄です。

NotebookLM